



VISUAL BASIC/ C# PROGRAMMING (330)

STATE – 2019

Production Portion:

Program 1: Matching Game _____ (400 points)

TOTAL POINTS _____ (***400 points***)

**Judge/Graders: Please double check and verify all
scores and answer keys!**

Property of Business Professionals of America.
May be reproduced only for use in the Business Professionals of America
Workplace Skills Assessment Program competition.



Matching Game

In this exercise, you will create a Visual Basic or C# Windows Form Application that requires the user to identify matching pairs of pictures. Upon hitting “Start Game”, the placement of pictures is visible for three (3) seconds, and then a default image is displayed.

Requirements:

1. You must create a C#/VB Windows Form Application named CS _330_ContestantNumber, where ContestantNumber is your BPA assigned contestant number (including dashes). For example, CS_330_01_2345_6789. If you are using VB, use VB instead of CS in the application name.
2. Your name and contestant number must appear as a comment at the top of the main source code file.
3. Application on Exit button press must ask for confirmation to close, with the Title of the confirmation box being “Confirm” and the text being “Are you sure you wish to exit the Matching Game?”
 - a. Exit button must be named “exitButton”
4. The main form must not have minimize or maximize, or close buttons in the Form Handle.
5. The main form must have text “Matching Game” for its title.
6. The main form must have a 3” x 4” placement of PictureBoxes.
 - a. Six images are provided. Each picture must be displayed twice to allow matching the same image.
 - b. The entire image must be visible within each PictureBox.
7. A Timer must be created and configured for a three (3) second interval.
8. The main form must have a “Start Game” button.
 - a. On click, the location of pictures must be randomized, the timer enabled, and score reset.
 - b. After three (3) seconds, the default image must be displayed for all PictureBoxes.
 - i. The timer event must only trigger once per “Start Button” click.
9. When a picture location is clicked, the image for that location is displayed (instead of the “default” image).
 - a. If two (2) images are matched correctly, the score increments by 100.
 - b. If two (2) images are matched incorrectly, the game is over.
 - i. Display a dialog stating final score.
 - ii. Allow images to be clicked to reveal, but do *not* allow score to change.



Sample Pictures:

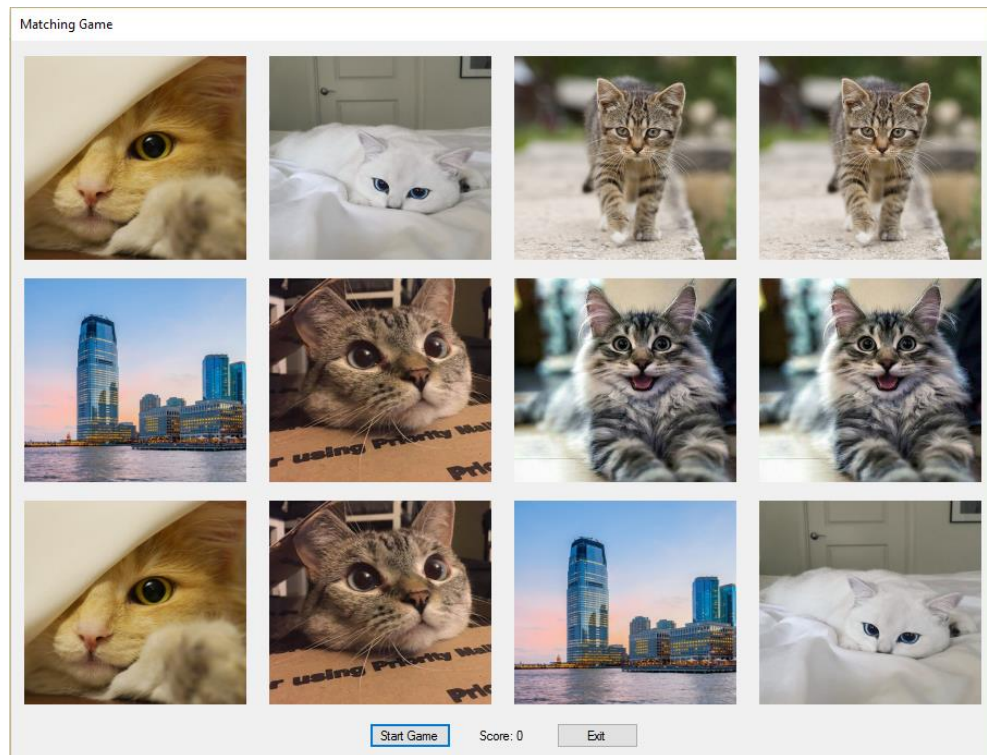


Figure 1: Main screen after form loads (randomized image locations).





Figure 2: The main form after “Start Game” is clicked and 3 seconds has elapsed.

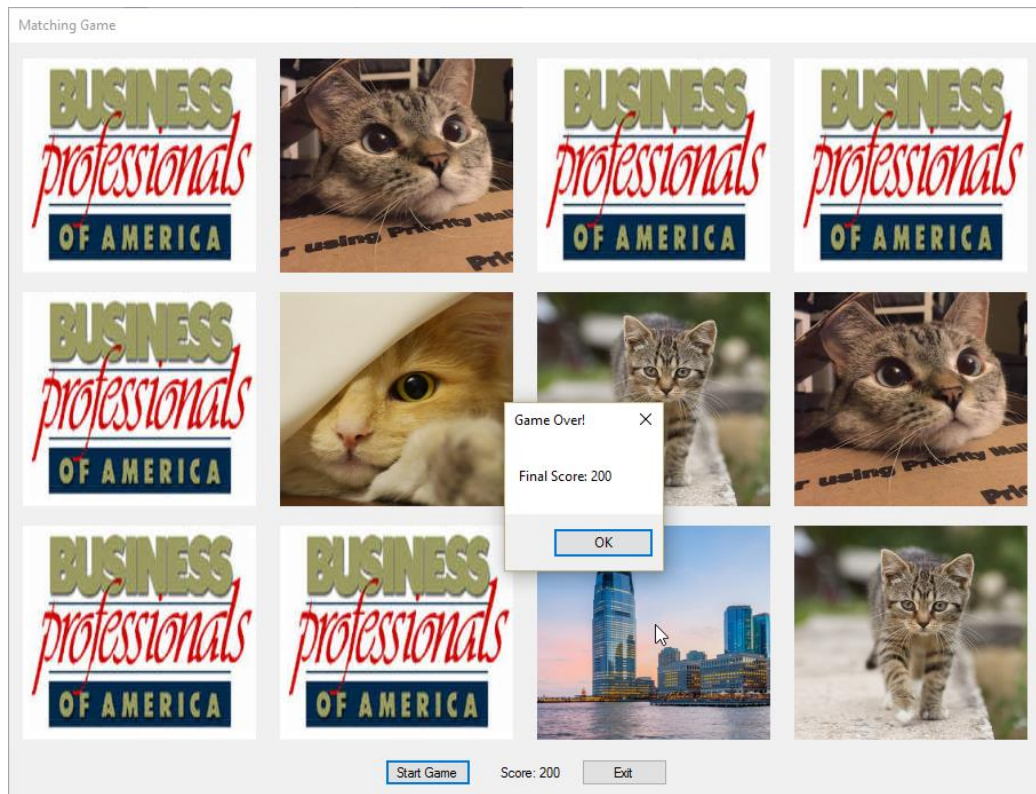


Figure 3: The main form after two correct matches and one incorrect match. Game Over dialog box also shown.

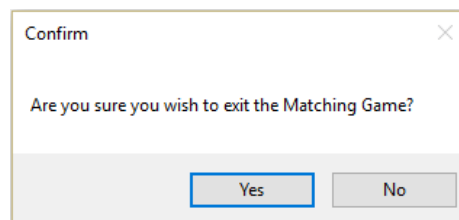


Figure 4: The confirmation window. This should be opened when Exit is pressed. If “Yes” is clicked, the application should close, otherwise return to the main form.

You will have ninety (90) minutes to complete your work.

Your name and/or school name should *not* appear on any work you submit for grading.

Submit a copy your entire solution/project to the flash drive provided. You must submit your entire solution/project so that the graders may open your project to review the source code and/or build and execute your solution/project. **Submissions that do not contain source code will not be graded.**

Development Standards

- Standard name prefixes must be utilized for variables.
- All subroutines, functions, and methods must be documented with comments explaining the purpose of the method, the input parameters (if any), and the output (if any).



Your application will be graded on the following criteria:

Solution and Project

The project is present on the flash drive _____ 10 points
The project is named according to the naming conventions _____ 10 points

Program Execution

Code copied to USB drive and program runs from USB _____ 20 points

If the program does not execute, then the remaining items in the program executive section receive a score of zero.

The program contains an Exit button that asks for user confirmation _____ 25 points
The exit button is named “exitButton” _____ 15 points
The main form does not have minimize, maximize, or close buttons
in the Form Handle _____ 20 points
The main form has 12 PictureBoxes with six (6) images displayed twice _____ 20 points
Start Game button randomizes pictures _____ 30 points
After Start Game, pictures are visible for three (3) seconds before default is shown _____ 30 points
During the initial three (3) seconds, selecting a picture is ignored _____ 20 points
Valid selections add 100 points _____ 20 points
Invalid selections display Game Over dialog box _____ 20 points
After Game Over, additional selections don’t add to score, but are allowed _____ 20 points
Start Game can be clicked at any time _____ 20 points

Source Code Review

Code is commented at the top, for each function, and as needed _____ 10 points
Code uses reasonable and consistent variable naming conventions _____ 10 points
Code to start game is present _____ 10 points
Code to display images for three (3) seconds is present _____ 20 points
Code to populate image boxes is present _____ 20 points
Code for creating/tracking time is present _____ 20 points
Code for displaying image when clicking pictureBox is present _____ 10 points
Code for randomly assigning images to pictureBoxes is present _____ 20 points

Total Points: 400 points

**SOURCE**

Form1.cs

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using System.Threading;

namespace matchinggame
{
    public partial class Form1 : Form
    {
        // Image to display when actual image is hidden
        string blankImage = "../../../BPALogo.jpg";

        // List of pictures to use for matching
        string[] pictures = { "../../../cats/cat1.jpg", "../../../cats/cat2.jpg",
        "../../../cats/cat3.jpg", "../../../cats/cat4.jpg", "../../../cats/cat5.jpg",
        "../../../city/city5.jpg" };

        List<PictureBox> pictureBoxes = new List<PictureBox>(); // List of all picture
boxes
        List<int> indices = new List<int>(); // Active picture indices
        List<int> selected = new List<int>(); // Selected indices.
        Expected to only increment to two before compare occurs
        bool isGameActive = false; // Indicates if game is
active
        int score = 0; // Game score

        public Form1()
        {
            InitializeComponent();

            // Add all picture boxes to a list
            pictureBoxes.Add(pictureBox1);
            pictureBoxes.Add(pictureBox2);
            pictureBoxes.Add(pictureBox3);
            pictureBoxes.Add(pictureBox4);
            pictureBoxes.Add(pictureBox5);
            pictureBoxes.Add(pictureBox6);
            pictureBoxes.Add(pictureBox7);
            pictureBoxes.Add(pictureBox8);
            pictureBoxes.Add(pictureBox9);
            pictureBoxes.Add(pictureBox10);
            pictureBoxes.Add(pictureBox11);
            pictureBoxes.Add(pictureBox12);

            // Populate 0-5, repeating each twice
            for (int i = 0; i < 12; i++)
            {
                indices.Add(((int)Math.Floor((double)i / 2.0)));
            }
        }
    }
}

```



```
// Set pictures on load
SetPictures();
}

#region Utility_Functions

// Shuffle active indices and set pictures
private void SetPictures()
{
    indices = Shuffle<int>(indices);
    for (int i = 0; i < 12; i++)
        pictureBoxes[i].Image = Image.FromFile(pictures[indices[i]]);
}

// Set all picture boxes to active picture
private void MakeAllVisible()
{
    for (int i = 0; i < 12; i++)
    {
        pictureBoxes[i].Image = Image.FromFile(pictures[indices[i]]);
    }
}

// Set all picture boxes to default picture
private void MakeAllInvisible()
{
    for (int i = 0; i < 12; i++)
        pictureBoxes[i].Image = Image.FromFile(blankImage);
}

// Check selected index for match
private void checkSelections(int i)
{
    // Add selected picture index to selected list
    selected.Add(i);

    // On second selection, check for match
    if (selected.Count == 2)
    {
        // Add to score for valid match
        if (indices[selected[0]] == indices[selected[1]])
        {
            SetScore(score + 100);
        }
        // End game and display Game over dialog for invalid match
        else
        {
            MessageBox.Show("Final Score: " + score.ToString(), "Game Over!");
            isGameActive = false;
        }
        selected.Clear();
    }
}

// Update score
private void SetScore(int newScore)
{
    score = newScore;
}
```



```

        scoreLabel.Text = "Score: " + score.ToString();
    }

    // Shuffle List
    private List<T> Shuffle<T>(List<T> list)
    {
        Random rng = new Random();
        int n = list.Count;
        while (n > 1)
        {
            n--;
            int k = rng.Next(n + 1);
            T value = list[k];
            list[k] = list[n];
            list[n] = value;
        }

        return list;
    }
#endregion

#region GUI_Events
private void startButton_Click(object sender, EventArgs e)
{
    isGameActive = false;
    SetScore(0);
    SetPictures();

    MakeAllVisible();
    timer1.Enabled = true;
}

private void exitButton_Click(object sender, EventArgs e)
{
    DialogResult r1 = MessageBox.Show("Are you sure you wish to exit the Matching
Game?", "Confirm", MessageBoxButtons.YesNo);
    if(r1 == DialogResult.Yes)
    {
        Application.Exit();
    }
}

private void timer1_Tick(object sender, EventArgs e)
{
    timer1.Enabled = false;
    MakeAllInvisible();
    isGameActive = true;
}

private void pictureBox_Click(object sender, EventArgs e)
{
    PictureBox clickedObj = (PictureBox)sender;
    int i = pictureBoxes.IndexOf(clickedObj);

    pictureBoxes[i].Image = Image.FromFile(pictures[indices[i]]);

    if (isGameActive)
    {
        checkSelections(i);
    }
}

```



```
        }  
    }  
}  
#endregion  
}
```

**Visual Basic Source Code Example**

```
Imports System
Imports System.Collections.Generic
Imports System.ComponentModel
Imports System.Data
Imports System.Drawing
Imports System.Linq
Imports System.Text
Imports System.Threading.Tasks
Imports System.Windows.Forms
Imports System.Threading
```

```
Namespace matchinggame
```

```
    Public Partial Class Form1
```

```
        Inherits Form
```

```
        Private blankImage As String = "../../../BPALogo.jpg"
```

```
        Private pictures As String() = {"../../../cats/cat1.jpg", "../../../cats/cat2.jpg", "../../../cats/cat3.jpg",
"../../../cats/cat4.jpg", "../../../cats/cat5.jpg", "../../../city/city5.jpg"}
```

```
        Private pictureBoxes As List(Of PictureBox) = New List(Of PictureBox)()
```

```
        Private indices As List(Of Integer) = New List(Of Integer)()
```

```
        Private selected As List(Of Integer) = New List(Of Integer)()
```

```
        Private isActive As Boolean = False
```

```
        Private score As Integer = 0
```

```
        Public Sub New()
```

```
            InitializeComponent()
```

```
            pictureBoxes.Add(pictureBox1)
```

```
            pictureBoxes.Add(pictureBox2)
```

```
            pictureBoxes.Add(pictureBox3)
```

```
            pictureBoxes.Add(pictureBox4)
```

```
            pictureBoxes.Add(pictureBox5)
```

```
            pictureBoxes.Add(pictureBox6)
```

```
            pictureBoxes.Add(pictureBox7)
```

```
            pictureBoxes.Add(pictureBox8)
```

```
            pictureBoxes.Add(pictureBox9)
```

```
            pictureBoxes.Add(pictureBox10)
```

```
            pictureBoxes.Add(pictureBox11)
```

```
            pictureBoxes.Add(pictureBox12)
```

```
        For i As Integer = 0 To 12 - 1
```

```
            indices.Add((CInt(Math.Floor(CDbl(i) / 2.0))))
```

```
        Next
```

```
        SetPictures()
```

```
    End Sub
```

```
    Private Sub SetPictures()
```

```
        indices = Shuffle(Of Integer)(indices)
```

```
        For i As Integer = 0 To 12 - 1
```

```
            pictureBoxes(i).Image = Image.FromFile(pictures(indices(i)))
```

```
        Next
```

```
    End Sub
```

```
    Private Sub MakeAllVisible()
```

```
        For i As Integer = 0 To 12 - 1
```



```

        pictureBoxes(i).Image = Image.FromFile(pictures(indices(i)))
    Next
End Sub

```

```

Private Sub MakeAllInvisible()
    For i As Integer = 0 To 12 - 1
        pictureBoxes(i).Image = Image.FromFile(blankImage)
    Next
End Sub

```

```

Private Sub checkSelections(ByVal i As Integer)
    selected.Add(i)

    If selected.Count = 2 Then

        If indices(selected(0)) = indices(selected(1)) Then
            SetScore(score + 100)
        Else
            MessageBox.Show("Final Score: " & score.ToString(), "Game Over!")
            isGameActive = False
        End If

        selected.Clear()
    End If
End Sub

```

```

Private Sub SetScore(ByVal newScore As Integer)
    score = newScore
    scoreLabel.Text = "Score: " & score.ToString()
End Sub

```

```

Private Function Shuffle(Of T)(ByVal list As List(Of T)) As List(Of T)
    Dim rng As Random = New Random()
    Dim n As Integer = list.Count

    While n > 1
        n -= 1
        Dim k As Integer = rng.[Next](n + 1)
        Dim value As T = list(k)
        list(k) = list(n)
        list(n) = value
    End While

    Return list
End Function

```

```

Private Sub startButton_Click(ByVal sender As Object, ByVal e As EventArgs)
    isGameActive = False
    SetScore(0)
    SetPictures()
    MakeAllVisible()
    timer1.Enabled = True
End Sub

```

```

Private Sub exitButton_Click(ByVal sender As Object, ByVal e As EventArgs)
    Dim r1 As DialogResult = MessageBox.Show("Are you sure you wish to exit the Matching Game?", "Confirm",
    MessageBoxButtons.YesNo)

```



```
If r1 = DialogResult.Yes Then
    Application.[Exit]()
End If
End Sub

Private Sub timer1_Tick(ByVal sender As Object, ByVal e As EventArgs)
    timer1.Enabled = False
    MakeAllInvisible()
    isGameActive = True
End Sub

Private Sub pictureBox_Click(ByVal sender As Object, ByVal e As EventArgs)
    Dim clickedObj As PictureBox = CType(sender, PictureBox)
    Dim i As Integer = pictureBoxes.IndexOf(clickedObj)
    pictureBoxes(i).Image = Image.FromFile(pictures(indices(i)))

    If isGameActive Then
        checkSelections(i)
    End If
End Sub
End Class
End Namespace
```